@Configuration vs @Component (with @Bean)

What Problem Do These Annotations Solve?

Spring needs to know:

Which Java classes should become objects inside the IoC container?

There are **two ways** to do this:

1. Let Spring **discover** classes automatically → @Component
2. Tell Spring **explicitly how to create objects** → @Configuration + @Bean

@Component — Automatic Bean Creation

What it means

@Component tells Spring:

"This class should be discovered during component scanning and turned into a bean."

Example

```java
@Component
public class PaymentService {
    public void pay() {
        System.out.println("Payment done");
    }
}
```

During startup:

1. Spring scans packages
2. Finds **@Component**
3. Creates object
4. Stores it in IoC container

You never control:

- How it is created
- What dependencies are passed → Spring does it for you.

@Configuration + @Bean — Explicit Bean Creation

@Configuration is used when:

You want to tell Spring exactly how an object must be created.

Example

@Configuration

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Now Spring does **not** discover PaymentService.

It executes:

paymentService()

and stores the returned object as a bean.

Why @Configuration Is NOT Just @Component

This is where most people fail interviews.

```
@Configuration
public class AppConfig {

    @Bean
    public Engine engine() {
        return new Engine();
    }

    @Bean
    public Car car() {
        return new Car(engine());
    }
}
```

You may think: **engine()** is called twice → two objects?

No.

Spring uses **CGLIB proxy** for @Configuration.

It intercepts:

engine() → and returns the same bean every time.

This guarantees **true singleton behavior**.

If you replace @Configuration with @Component:

@Component

public class AppConfig { ... }

Now:

engine()

is called directly → **new object created every time** → broken singleton.

This is a **very common interview trap**.

When to Use Which

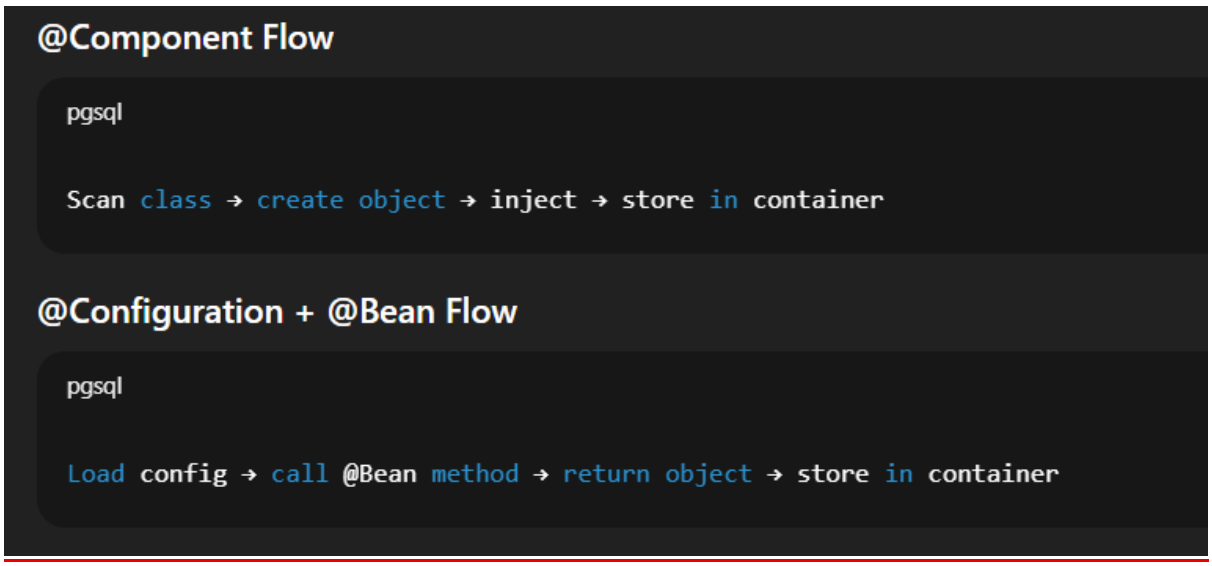
Situation	Use
Your own classes	@Component, @Service, @Repository
Third-party classes	@Configuration + @Bean
Complex object creation	@Configuration + @Bean
Simple POJOs	@Component

Example:
You cannot put @Component on a library class like DataSource.
So you must use:

@Bean

public DataSource dataSource() { }

Flow Comparison



CGLIB Proxy – What Really Happens in @Configuration

CGLIB = Code Generation Library

In Spring:

CGLIB Proxy = a runtime-generated subclass of your class that intercepts method calls

In simple words:

Spring creates a fake child class of your class and puts logic in between.

When you write:

```
@Configuration
public class AppConfig {

    @Bean
    public Engine engine() {
        return new Engine();
    }

    @Bean
    public Car car() {
        return new Car(engine());
    }
}
```

Spring does **not** use AppConfig directly.

It creates a **proxy subclass** using CGLIB.

Conceptually:

AppConfig ---> AppConfig$$SpringCGLIBProxy

Every call to a @Bean method goes through this proxy.

What the proxy actually does

When car() calls engine():

```
car() → proxy.engine()
```

The proxy checks:

```
Is Engine already in IoC container?
    Yes → return existing bean
    No  → create it, store it, then return it
```

So even though your Java code looks like this:

```
return new Car(engine());
```

What actually happens is:

```
return new Car( getBean("engine") );
```

This is why @Configuration guarantees:

- Singleton behavior
- No duplicate objects
- Correct dependency graph

If you use @Component instead

```java
@Component
public class AppConfig {

    @Bean
    public Engine engine() {
        return new Engine();
    }

    @Bean
    public Car car() {
        return new Car(engine()); // direct call
    }
}
```

Now there is **no proxy**.

So:

```
engine() → new Engine()
engine() → new Engine()   (again)
```

Two different objects → broken singleton → production bugs.

This is one of the most common Spring interview traps.

Bean Creation Cheat-Sheet

Using @Component

```
Application starts
        ↓
ComponentScan runs
        ↓
Finds @Component classes
        ↓
Creates object using constructor
        ↓
Injects dependencies
        ↓
Stores in IoC container
        ↓
Bean ready to use
```

You never control object creation.

Spring decides everything.

Using @Configuration + @Bean

```
Application starts
        ↓
@Configuration class loaded
        ↓
Spring creates CGLIB proxy
        ↓
@Bean methods are registered
        ↓
When bean is needed:
    → Call @Bean method via proxy
    → Create object (if not already created)
    → Store in IoC container
        ↓
Return same instance every time
```

Here **you control construction**, Spring controls lifecycle.

When to use what (Interview Ready)

Scenario	Use
Your own service, controller, repository	@Component, @Service, @Repository
Third-party class (DataSource, Kafka, SDKs)	@Bean
Complex object creation logic	@Bean
Need to pass constructor params	@Bean
Need conditional creation	@Bean

One-Line Memory Hook

@Component lets Spring discover beans.
@Configuration + @Bean lets you design how beans are built.

Interview-Critical Questions

Q1. Difference between @Component and @Configuration?
@Component marks a class for auto-detection.
@Configuration defines how beans are created explicitly.

Q2. Why does Spring use CGLIB with @Configuration?
To ensure @Bean methods return singleton instances.

Q3. Can @Bean work without @Configuration?
Yes, but singleton behaviour is not guaranteed.

Q4. When should I prefer @Bean over @Component?
When you don't own the class or need custom construction.

One-Line Interview Summary

"@Component lets Spring discover beans, while @Configuration + @Bean lets you define exactly how beans are created, with proxy-based singleton guarantees."